

Lezione 1 - Introduzione

Prerequisiti

Il corso assume conoscenze di base di programmazione in **C**, in particolare:

- puntatori
- allocazione dinamica della memoria
- strutture
- definizione di funzioni

È richiesta anche una conoscenza base di **Python**.

Sono inoltre necessari concetti di *architettura dei calcolatori*, sistemi operativi, multithreading, algoritmi e strutture dati.

Esame

L'esame è composto da **progetto**, **scritto** e **orale**.

- progetto → **50%**
- scritto + orale → **50%**

Per accedere all'orale è necessario superare sia progetto che scritto.

Il progetto è individuale ed è diviso in due parti:

- una in **C**
- una in **Python**

Struttura del calcolatore

Un programma viene eseguito dalla CPU attraverso un ciclo continuo:

richiesta → decodifica → esecuzione

Questo è il modello base, ma nelle CPU moderne il processo è molto più complesso e permette di eseguire più istruzioni contemporaneamente.

Accesso alla memoria

A questo punto nasce un problema fondamentale: la **memoria (RAM)** è molto più lenta della CPU.

Se la CPU dovesse aspettare ogni accesso alla memoria, l'esecuzione diventerebbe molto lenta.

Per questo esiste una gerarchia di **cache**:

- **L1** → molto veloce, molto piccola
- **L2**
- **L3** → più grande ma più lenta

I dati vengono trasferiti a **blocchi**, quindi accedere a indirizzi vicini è più efficiente rispetto ad accedere a indirizzi lontani.

Questo comportamento porta al concetto di **località di memoria**

Località di memoria

La memoria non viene accessa un elemento alla volta, ma a **blocchi**.

Questo significa che quando leggiamo un dato, in realtà vengono caricati anche i dati vicini.

Di conseguenza:

- accessi a celle **vicine** → più veloci
- accessi a celle **lontane** → più lenti

Questo concetto è chiamato *località di memoria*.

Accessi con stride

Per capire meglio questo comportamento, consideriamo l'accesso a un array.

Se accediamo in modo consecutivo (`stride = 1`), sfruttiamo la cache e l'esecuzione è veloce.

Se invece accediamo con salti (`stride = 2, 4, 8, ...`), perdiamo la località di memoria e le prestazioni peggiorano.

Anche se il numero di operazioni è lo stesso, il tempo cambia.



Codice

```
int readmem(char * m, int num, int stride)
{
    int sum = 0;

    for (int i = 0; i < num; i++)
    {
        sum += m[stride * i]; // Accesso con salto nella memoria
    }

    return sum;
}
```

👉 `stride` determina la distanza tra gli accessi:

- `stride = 1` → accessi consecutivi → **cache efficiente**
- `stride` grande → accessi distanti → cache inefficiente → più lento



Branch prediction

Le CPU moderne non eseguono una sola istruzione alla volta, ma cercano di eseguire più istruzioni contemporaneamente.

Quando incontrano un'istruzione condizionale (`if`), devono decidere quale ramo eseguire. Per migliorare le prestazioni, la CPU prova a **prevedere** il risultato della condizione (*branch prediction*).

Se la previsione è corretta, l'esecuzione è veloce.

Se è sbagliata, la CPU deve annullare il lavoro fatto e questo causa un rallentamento.

Questo dipende dai dati:

- dati ordinati → comportamento prevedibile → più veloce
- dati casuali → comportamento imprevedibile → più lento



Codice

```
int sum_small_values(int * v, int len)
{
    int sum = 0;

    for (int i = 0; i < len; i++)
    {
        if (v[i] < 50)    // Condizione che la CPU cerca di prevedere
        {
            sum += v[i];    // Eseguito solo se la condizione è vera
        }
    }

    return sum;
}
```

👉 il tempo dipende da quanto la condizione è prevedibile:

- sequenza ordinata → CPU impara → veloce
- sequenza casuale → CPU sbaglia spesso → più lento



Moltiplicazione di matrici

Consideriamo il problema della moltiplicazione di matrici.

L'algoritmo classico ha complessità **$O(n^3)$** , quindi il numero di operazioni cresce molto rapidamente all'aumentare della dimensione.

Tuttavia, anche a parità di complessità teorica, le prestazioni possono essere molto diverse a seconda dell'implementazione.



Codice C

```
C[i][j] += A[i][k] * B[k][j]; // Prodotto riga per colonna
```

👉 triplo ciclo annidato → complessità $O(n^3)$



Codice Python

```
def matmul(A, B, C):  
    for i in range(0, len(A)):  
        for j in range(0, len(B[0])):  
            for k in range(0, len(A[0])):  
                C[i][j] += A[i][k] * B[k][j]
```

👉 stesso algoritmo → più lento (interprete)



NumPy

```
c = np.dot(a, b)
```

👉 molto più veloce perché:

- usa codice in **C**
- sfrutta librerie ottimizzate (*BLAS*)
- utilizza meglio **cache** e CPU



Idea fondamentale

A questo punto il messaggio della lezione è chiaro:

le performance non dipendono solo dall'algoritmo, ma anche da come viene implementato

In particolare dipendono da:

- accesso alla memoria
- uso della **cache**
- *branch prediction*
- ottimizzazioni e librerie utilizzate